

UNIVERSIDAD TRES CULTURAS
INGENIERIA EN SISTEMAS COMPUTACIONALES

www.utc.mx

"VERIFEX"

Analizador de Credibilidad de Noticias
con Inteligencia Artificial Local

TESIS

QUE PARA OBTENER EL TITULO DE INGENIERO EN
SISTEMAS COMPUTACIONALES PRESENTAN:

Chapa Tinajero Francisco Yahel

Fernandez Casas Carlos Axel

Gallardo Cortes Valeria

Garcia Garcia Jose Armando

ASESORES:

M. en A. T. Gerardo Estrada Gutierrez

M. en E. E. Erika Arellano Orozco

Ciudad de Mexico, agosto 2025

CARTA DE ACEPTACION

Los abajo firmantes, miembros del Comité de Tesis de la Universidad Tres Culturas, hacemos constar que hemos revisado y aprobado la tesis titulada "VERIFEX: Analizador de Credibilidad de Noticias con Inteligencia Artificial Local", presentada por los pasantes Chapa Tinajero Francisco Yahel, Fernandez Casas Carlos Axel, Gallardo Cortes Valeria y Garcia Garcia Jose Armando, para obtener el título de Ingeniero en Sistemas Computacionales.

Fecha: _____

M. en A. T. Gerardo Estrada Gutierrez
Asesor

M. en E. E. Erika Arellano Orozco
Asesor

Sinodal

Sinodal

AGRADECIMIENTOS

Chapa Tinajero Francisco Yahel:

Quiero expresar mi mas profundo agradecimiento a mi familia, quienes han sido mi apoyo incondicional durante toda mi formacion academica. A mi padre Francisco, por su ejemplo de perseverancia y dedicacion. A mi madre Lidia, que desde el cielo guia mis pasos. A mis hermanos Karina, Hugo y Naomi, por su apoyo incondicional. A mi novia Monse, por su paciencia y carino durante este proceso. A mis amigos y companeros, gracias por hacer de esta travesia universitaria una experiencia inolvidable. Y por supuesto, a mis asesores, el Mtro. Gerardo Estrada y la Mtra. Erika Arellano, por su guia y conocimientos compartidos en la realizacion de esta tesis.

Fernandez Casas Carlos Axel:

Agradezco profundamente a mis asesores, Gerardo Estrada y Erika Arellano, por su guia y apoyo inquebrantables durante el desarrollo de esta investigacion. Su conocimiento, paciencia y dedicacion han sido invaluableles. Extiendo mi gratitud a los miembros del comite por sus valiosas contribuciones. A mi familia, especialmente a mis padres, por su amor y apoyo incondicional. Sin su confianza y sacrificio, este logro no habria sido posible. A mis amigos y companeros, gracias por su constante apoyo y animo.

Gallardo Cortes Valeria:

Quiero agradecer a mi familia, que ha sido mi pilar incondicional. A mis padres Leticia y Diego, por su amor, apoyo y sacrificios de cada dia. Ustedes siempre han creido en mi y me han ensenado la importancia del esfuerzo y la perseverancia. A mis hermanos, por su paciencia. A mis amigos, quienes han estado a mi lado durante todo este proceso. Agradezco a mis asesores por compartir sus conocimientos y guiarme en el camino a la formacion de esta tesis. Cada uno de ustedes ha dejado una huella en mi vida.

Garcia Garcia Jose Armando:

Inicio mi agradecimiento a todos los que formaron parte de este largo proceso. A mis asesores Erika Arellano Orozco y Gerardo Estrada Gutierrez por haberme guiado en este proyecto y por su involucramiento en la investigacion que hoy presento. A mis padres, mis hermanos y todos mis familiares, gracias por apoyarme incondicionalmente. Su aliento y comprension fueron fundamentales. Finalmente, agradezco a los usuarios que se tomen el tiempo de utilizar VERIFEX. Espero que encuentren en esta herramienta una forma de combatir la desinformacion.

TABLA DE CONTENIDO

INTRODUCCION

CAPITULO 1: GENERALIDADES DEL PROYECTO

- 1.1 Planteamiento del problema
- 1.2 Objetivos
 - 1.2.1 General
 - 1.2.2 Especificos
- 1.3 Justificacion
- 1.4 Alcances
- 1.5 Limitaciones

CAPITULO 2: FUNDAMENTO TEORICO

- 2.1 Antecedentes del problema
- 2.2 Definiciones de terminos
- 2.3 Marco teorico
 - 2.3.1 Ambiental
 - 2.3.2 Economico
 - 2.3.3 Tecnologico
 - 2.3.4 Social
- 2.4 Estudio de mercado
- 2.5 Viabilidad
 - 2.5.1 Riesgos
 - 2.5.2 Plan de contingencia
- 2.6 Factibilidad

CAPITULO 3: METODOLOGIA DE DESARROLLO

- 3.1 Metodologia Kanban
- 3.2 Fase 1: Gestion del Backlog (Requerimientos)
 - 3.2.1 Levantamiento de requerimientos
 - 3.2.2 Descripcion del proceso de negocio
 - 3.2.3 Historias de usuario
 - 3.2.4 Requerimientos no funcionales
 - 3.2.5 Requerimientos del sistema
 - 3.2.6 Estudio de usabilidad
- 3.3 Fase 2: Diseno y Arquitectura
 - 3.3.1 Arquitectura del sistema
 - 3.3.2 Diagrama de casos de uso
 - 3.3.3 Diagrama de actividades
 - 3.3.4 Diagrama de secuencia

3.3.5 Diagrama de clases

3.3.6 Diagrama entidad-relacion

3.3.7 Wireframes

3.4 Fase 3: Implementacion (Flujo Kanban)

3.4.1 Historial de versiones del sistema

3.5 Fase 4: Codificacion

CAPITULO 4: IMPLEMENTACION Y RESULTADOS

4.1 Fase de pruebas

4.1.1 Pruebas del sistema

4.1.1.1 Pruebas de validacion

4.1.1.2 Pruebas de funcionalidad

4.1.2 Pruebas de usabilidad

CONCLUSIONES

ANEXO A: MANUAL TECNICO

ANEXO B: MANUAL DE USUARIO

ANEXO C: CODIFICACION

INTRODUCCION

En un mundo digital donde la informacion fluye a una velocidad sin precedentes, la desinformacion y las noticias falsas se han convertido en uno de los desafios mas significativos de la sociedad contemporanea. La facilidad con la que el contenido enganoso puede crearse y difundirse a traves de redes sociales y sitios web ha generado una crisis de credibilidad que afecta a millones de personas en todo el mundo, incluyendo Mexico y America Latina.

La sociedad actual enfrenta desafios considerables en la identificacion de informacion veraz. Estudios recientes indican que mas del 60% de la poblacion mexicana ha encontrado noticias falsas en redes sociales, y una parte significativa admite haberlas compartido sin verificar su autenticidad. Esta situacion revela una brecha importante en las herramientas digitales disponibles para la verificacion de contenido informativo.

A medida que la desinformacion impacta la salud publica, la democracia y la cohesion social, surge la necesidad de proporcionar herramientas accesibles que permitan a los ciudadanos comunes verificar la credibilidad de las noticias que consumen. En este contexto, se presenta una solucion innovadora: VERIFEX, un analizador de credibilidad de noticias que utiliza inteligencia artificial local para evaluar la veracidad del contenido informativo.

VERIFEX es una aplicacion web que funciona completamente en el equipo del usuario sin necesidad de conexion a internet (excepto para la obtencion de noticias similares). Utiliza el motor de IA local Ollama con modelos como llama3.2:1b para analizar el contenido de las noticias, extraer afirmaciones principales, identificar banderas rojas y senales positivas, y proporcionar un veredicto de credibilidad con un nivel de confianza.

La aplicacion esta disenada para ser accesible a cualquier persona con conocimientos basicos de navegacion web. A traves de una interfaz sencilla e intuitiva con estetica cyberpunk, el usuario simplemente pega una URL de una noticia y obtiene un analisis detallado que incluye: veredicto (REAL, FALSO, SATIRA, ESTAFA o NO VERIFICABLE), indice de confiabilidad, resumen del articulo, afirmaciones principales, analisis detallado, alertas detectadas, senales positivas y noticias similares relacionadas.

El proposito fundamental de VERIFEX es empoderar a los ciudadanos en la lucha contra la desinformacion, proporcionando una herramienta gratuita, privada (todo el analisis ocurre localmente) y accesible que permita tomar decisiones informadas sobre el contenido que se consume y comparte en el entorno digital.

CAPITULO 1

GENERALIDADES DEL PROYECTO

Para el desarrollo de VERIFEX, un analizador de credibilidad de noticias basado en inteligencia artificial local, se abarcan los siguientes puntos que se contemplan en este primer capitulo:

- a) Planteamiento del problema
- b) Definicion de objetivos
- c) Establecimiento de alcances
- d) Identificacion de limitaciones
- e) Justificar la necesidad del proyecto para contar con una base solida para su ejecucion.

Es importante tener en cuenta estos elementos para asegurar un desarrollo efectivo y exitoso del analizador de credibilidad.

1.1. Planteamiento del problema

En la actualidad, las noticias falsas y la desinformacion se han convertido en un fenomeno global que afecta significativamente a la sociedad. La facilidad con la que se crea y distribuye contenido enganoso a traves de plataformas digitales ha generado una crisis de confianza en los medios de comunicacion y en la informacion que circula en internet.

En Mexico, se estima que aproximadamente el 70% de la poblacion tiene acceso a internet, y de estos, una gran mayoria consume noticias a traves de redes sociales y sitios web. Sin embargo, la capacidad para distinguir entre informacion veraz y falsa no ha crecido al mismo ritmo que el volumen de informacion disponible. Esto ha llevado a que las noticias falsas se difundan hasta seis veces mas rapido que las noticias verdaderas en plataformas digitales.

Actualmente existen diversas herramientas y plataformas dedicadas a la verificacion de hechos (fact-checking), como Verificado, Animal Politico, Snopes y Google Fact Check. Sin embargo, estas herramientas presentan limitaciones significativas: muchas requieren conexion constante a internet, dependen de API keys o servicios de pago, no funcionan de manera local, o estan disenadas para periodistas y no para el publico general.

La investigacion preliminar para la creacion de VERIFEX revelo una brecha significativa en las herramientas digitales disponibles para la verificacion de noticias. Aunque se identificaron diversas aplicaciones y servicios relacionados con el fact-checking, estos carecian de un enfoque de accesibilidad universal, centrandose en suscripciones de pago, dependencia de servicios en la nube, o requiriendo conocimientos tecnicos avanzados para

su utilizacion.

Por lo tanto, surge la necesidad de abordar esta carencia mediante la creacion de una herramienta mas efectiva, accesible y, sobre todo, que funcione de manera local sin depender de servicios externos. El proyecto de tesis VERIFEX se propone desarrollar un analizador de credibilidad de noticias que aborde esta necesidad, ofreciendo una solucion gratuita, privada y eficiente.

En este contexto, el planteamiento del problema se enfoca en la siguiente interrogante:

Como puede una herramienta de analisis de credibilidad basada en inteligencia artificial local, como la propuesta en el proyecto VERIFEX, mejorar la capacidad de los usuarios para identificar noticias falsas y desinformacion en el entorno digital mexicano, considerando las limitaciones de las herramientas existentes?

1.2. Objetivos

A continuacion, se presentan los objetivos divididos en general y en especificos.

1.2.1. General

Desarrollar un analizador de credibilidad de noticias utilizando inteligencia artificial local que permita a los usuarios verificar la veracidad de contenido informativo en linea, de forma eficaz, gratuita y accesible para el publico en general en Mexico y America Latina.

1.2.2. Especificos

Disenar una interfaz de usuario intuitiva y atractiva que permita a los usuarios analizar URLs de noticias de forma sencilla, con estetica cyberpunk y elementos visuales que faciliten la comprension de los resultados.

Implementar un modulo de extraccion de contenido (scraping) que obtenga el texto principal de articulos de noticias eliminando elementos de ruido como publicidad y navegacion.

Integrar un motor de inteligencia artificial local (Ollama) para analizar el contenido extraido y clasificarlo en las categorias: REAL, FALSO, SATIRA, ESTAFA o NO VERIFICABLE.

Desarrollar un sistema de busqueda de noticias similares utilizando Google News RSS para proporcionar contexto adicional al usuario.

Implementar soporte bilingue (espanol e ingles) para hacer la herramienta accesible a una audiencia mas amplia.

1.3. Justificacion

Tal y como se comentaba en el apartado de planteamiento del problema, se buscaba realizar una herramienta que estuviera enfocada a combatir la desinformacion. Es decir, se buscaba un enfoque donde ademas de ser funcional

y accesible, tuviera un impacto social positivo. Entre las investigaciones preliminares y la observacion del panorama actual de las noticias falsas en Mexico, se identifico que existe un conocimiento limitado sobre como verificar la informacion que se consume diariamente.

Se realizo una investigacion preliminar para identificar herramientas existentes de verificacion de noticias, encontrando que las principales opciones son servicios en linea que requieren conexion a internet y muchas veces dependen de suscripciones o donaciones. Herramientas como Verificado, Google Fact Check Tool y plataformas internacionales como Snopes o Politifact ofrecen servicios valiosos, pero presentan limitaciones en cuanto a disponibilidad local, idioma, y accesibilidad para el usuario promedio.

Al no haber muchas herramientas accesibles, gratuitas y que funcionen de manera local para la verificacion de noticias, se genero la idea del proyecto VERIFEX. La carencia de informacion y herramientas de verificacion contribuye a la propagacion de desinformacion, por lo que VERIFEX se propone desarrollar un analizador de credibilidad con las siguientes características:

Funcionamiento 100% local: todo el analisis se realiza en el equipo del usuario sin enviar datos a servidores externos.

Gratuito: utiliza software de codigo abierto y modelos de IA locales sin costo.

Privacidad: al ser local, no se almacena ni comparte informacion del usuario.

Accesible: interfaz sencilla e intuitiva disenada para el publico general.

Bilingue: soporte para espanol e ingles.

Esta iniciativa es vital para abordar la falta de herramientas accesibles de verificacion de noticias y contribuir a una mejor alfabetizacion mediatica en la sociedad mexicana.

1.4. Alcances

VERIFEX debe ser capaz de analizar la credibilidad de noticias a traves de su URL, extrayendo el contenido principal del articulo.

Debe clasificar el contenido en cinco categorias: REAL, FALSO, SATIRA, ESTAFA y NO VERIFICABLE, proporcionando un nivel de confianza asociado.

Debe extraer las afirmaciones principales del articulo y presentar un analisis detallado con las razones detras del veredicto.

Debe identificar banderas rojas (senales de alerta) y senales positivas en el contenido analizado.

Debe buscar noticias similares en Google News para proporcionar contexto adicional.

La interfaz de usuario se disenara de manera intuitiva y atractiva con estetica cyberpunk, facilitando la comprension de los resultados.

Desarrollo del backend utilizando Python con Flask y el frontend con React, TypeScript y Vite.

Proporcionara soporte bilingue (espanol e ingles) para mayor accesibilidad.

1.5. Limitaciones

A continuacion, se presentan las limitaciones que se han identificado en el proyecto:

La herramienta requiere que el usuario tenga instalado Node.js, Python 3.10+ y Ollama en su equipo, lo que puede representar una barrera tecnica para usuarios no familiarizados con estas tecnologias.

VERIFEX no es compatible con sistemas operativos inferiores a Windows 10, macOS 10.15 o distribuciones de Linux sin soporte para las dependencias necesarias.

La aplicacion no analiza contenido multimedia como videos, imagenes o audio; se limita al contenido textual extraido de paginas web.

La precision del analisis depende del modelo de inteligencia artificial utilizado (por defecto llama3.2:1b) y puede variar segun la complejidad del contenido.

No se almacena ningun dato del usuario; cada analisis es completamente en memoria y se descarta al cerrar la aplicacion.

La busqueda de noticias similares requiere conexion a internet para acceder a Google News RSS.

CAPITULO 2

FUNDAMENTO TEORICO

En este capitulo se abarca la base teorica necesaria para comprender el desarrollo de VERIFEX. Se contemplan los temas que seran necesarios dominar para la ejecucion del analizador de credibilidad, el estado del arte, estudio de mercado, tablas de viabilidad y riesgos, ademas del glosario de terminos con las palabras mas destacadas a lo largo de la documentacion.

2.1. Antecedentes del problema

Para el desarrollo de este proyecto se considero crear una herramienta de analisis de credibilidad de noticias. De acuerdo con la siguiente informacion podemos tener un mejor contexto sobre el tema.

- Verificado MX (Mexico, 2018):

Verificado MX es una iniciativa de fact-checking mexicana que surgio como respuesta a la desinformacion durante el proceso electoral de 2018. Esta alianza entre medios de comunicacion, organizaciones civiles y universidades se ha mantenido activa verificando contenido viral y noticias falsas. Aunque es un esfuerzo valioso, opera principalmente a traves de un sitio web y redes sociales donde los usuarios deben esperar la verificacion manual por parte de periodistas, lo que limita su capacidad de respuesta en tiempo real.

- Google Fact Check Tools (Global, 2017):

Google lanzo Fact Check Tools como parte de su iniciativa contra la desinformacion. Esta herramienta permite buscar verificaciones de hechos realizadas por organizaciones de fact-checking en todo el mundo. Sin embargo, depende totalmente de que organizaciones externas hayan verificado previamente un contenido, y no realiza analisis automatico del contenido.

- Snopes y Politifact (Estados Unidos, 1994/2007):

Snopes y Politifact son dos de las plataformas de verificacion de hechos mas reconocidas a nivel mundial. Snopes se enfoca en desmentir rumores y leyendas urbanas, mientras que Politifact verifica declaraciones de politicos. Ambas plataformas requieren verificacion manual por parte de periodistas y no ofrecen analisis automatico de contenido.

- Proyectos de deteccion automatica de fake news con IA:

En el ambito academico y tecnologico, diversos proyectos han explorado el uso de inteligencia artificial para la deteccion automatica de noticias falsas. Estos incluyen sistemas basados en procesamiento de lenguaje natural

(NLP), aprendizaje automatico y analisis de redes. Sin embargo, la mayoría de estos proyectos requieren infraestructura en la nube, grandes conjuntos de datos y recursos computacionales significativos, lo que limita su accesibilidad para el usuario comun.

2.2. Definiciones de terminos

Este apartado tiene como objetivo asegurar que el lector tenga una comprension unificada y correcta de los terminos especializados que se presentan en el documento.

Desinformacion:

Informacion falsa o enganosa creada y difundida intencionalmente para enganar a las personas.

Noticias falsas (Fake News):

Contenido informativo que imita el formato de noticias legitimas pero contiene informacion falsa, fabricada o enganosa.

Fact-checking:

Proceso de verificacion de hechos y afirmaciones contenidas en contenido informativo para determinar su veracidad.

Credibilidad:

Cualidad de ser creible o digno de confianza. En el contexto de VERIFEX, se refiere a la probabilidad de que una noticia sea veraz.

Inteligencia Artificial Local:

Modelos de IA que se ejecutan directamente en el equipo del usuario sin necesidad de conexion a servidores externos.

Ollama:

Motor de inteligencia artificial local que permite ejecutar modelos de lenguaje como llama3.2:1b en el equipo del usuario.

Procesamiento de Lenguaje Natural (NLP):

Rama de la inteligencia artificial que se ocupa de la interaccion entre computadoras y el lenguaje humano.

Web Scraping:

Tecnica utilizada para extraer datos de sitios web de forma automatica.

Frontend:

Parte de un sistema informatico que se encarga de la interaccion directa con el usuario y la presentacion de informacion.

Backend:

Parte de un sistema informatico que se encarga de procesar y almacenar datos, sin que el usuario tenga acceso

directo a ellos.

API:

Interfaz de Programacion de Aplicaciones, conjunto de definiciones y protocolos para integrar servicios de software.

Framework:

Entorno de trabajo que proporciona herramientas y funcionalidades para facilitar el desarrollo de software.

React:

Biblioteca de JavaScript para construir interfaces de usuario, desarrollada por Facebook.

TypeScript:

Superconjunto de JavaScript que anade tipado estatico opcional al lenguaje.

Flask:

Framework minimalista de Python para desarrollo de aplicaciones web.

Vite:

Herramienta de construccion rapida para proyectos web modernos.

BeautifulSoup:

Biblioteca de Python para extraer datos de archivos HTML y XML.

2.3. Marco teorico

En esta seccion se presentan los fundamentos teoricos que respaldan el desarrollo de VERIFEX, un analizador de credibilidad de noticias basado en inteligencia artificial local. El marco teorico esta dividido en cuatro secciones: ambiental, economico, tecnologico y social.

2.3.1. Ambiental

El ambito ambiental del desarrollo de software se centra en la urgente necesidad de utilizar la tecnologia de forma responsable y sostenible para minimizar el impacto negativo en nuestro planeta. La eficiencia energetica y la reduccion de la huella de carbono son dos pilares fundamentales en este aspecto.

VERIFEX contribuye positivamente al ambito ambiental al ejecutar todo el procesamiento de forma local, eliminando la necesidad de infraestructura en la nube que consume grandes cantidades de energia. Los modelos de IA locales, aunque requieren recursos computacionales, evitan la transmision de datos a traves de internet y el uso de centros de datos masivos.

Al optimizar el consumo de energia mediante el uso de modelos ligeros como llama3.2:1b y practicas de desarrollo eficientes, se pueden tomar decisiones informadas y responsables para garantizar un equilibrio sostenible entre el progreso tecnologico y la proteccion del medio ambiente.

2.3.2. Economico

El desarrollo de VERIFEX implica una serie de consideraciones economicas que han sido minimizadas gracias a la utilizacion de herramientas y software gratuitos o de codigo abierto. De esta manera se puede mantener un margen de inversion bajo y cumplir con el objetivo de crear un analizador de credibilidad accesible para todos.

A continuacion, se comentan de forma breve los softwares y herramientas que se utilizaron:

React + TypeScript + Vite:

React es una biblioteca de codigo abierto mantenida por Meta (Facebook) y una comunidad de desarrolladores. Vite es una herramienta de construccion gratuita y de codigo abierto. Su uso elimina la necesidad de costosas licencias de software para el desarrollo del frontend.

Python + Flask:

Python es un lenguaje de programacion de codigo abierto, y Flask es un framework web minimalista gratuito. Ambos permiten el desarrollo del backend sin incurrir en costos de licencia.

Ollama:

Ollama es un motor de IA local gratuito y de codigo abierto que permite ejecutar modelos de lenguaje en el equipo del usuario sin necesidad de servicios en la nube ni API keys de pago.

BeautifulSoup + lxml:

Bibliotecas de Python gratuitas y de codigo abierto para el web scraping y procesamiento de HTML.

Estas herramientas permitieron reducir significativamente la inversion necesaria para desarrollar el analizador de credibilidad. Ademas, el equipo ya contaba con computadoras capaces de manejar las demandas del desarrollo, lo que elimino la necesidad de inversiones adicionales en hardware.

2.3.3. Tecnologico

Dentro del ambito tecnologico, es esencial adquirir ciertos conocimientos fundamentales para llevar a cabo el desarrollo del analizador de credibilidad de manera efectiva. En este apartado se exploraran temas como frameworks de desarrollo, lenguajes de programacion, inteligencia artificial local y diseno de interfaces.

Arquitectura del sistema:

VERIFEX adopta una arquitectura cliente-servidor donde el frontend (React + TypeScript) se comunica con el backend (Python + Flask) a traves de una API REST. El backend se encarga de realizar el scraping de la URL, analizar el contenido mediante Ollama, y buscar noticias similares en Google News.

Frontend - React con TypeScript:

React es una biblioteca de JavaScript para construir interfaces de usuario basadas en componentes. Su enfoque

declarativo permite crear interfaces interactivas de manera eficiente. TypeScript anade tipado estatico que mejora la robustez del codigo y facilita el mantenimiento. Vite se utiliza como herramienta de construccion por su rapidez y compatibilidad con proyectos modernos.

Backend - Python con Flask:

Python es un lenguaje de programacion versatil y ampliamente utilizado en ciencia de datos y desarrollo web. Flask es un framework minimalista que permite crear aplicaciones web de forma rapida y eficiente. Se eligio Flask por su simplicidad y flexibilidad, ideal para un proyecto de esta escala.

Inteligencia Artificial Local - Ollama:

Ollama es un motor de IA que permite ejecutar modelos de lenguaje grandes (LLMs) de forma local en el equipo del usuario. VERIFEX utiliza el modelo llama3.2:1b, que ofrece un equilibrio optimo entre rendimiento y precision para la tarea de clasificacion de credibilidad. La IA local ofrece ventajas significativas: privacidad de datos, funcionamiento sin internet, sin costos de API y baja latencia.

Diseño de interfaz:

El diseño de interfaz de VERIFEX sigue una estetica cyberpunk con colores oscuros, acentos en cian y rojo, tipografia monospace para elementos tecnicos y efectos visuales como glitch y scanlines. La interfaz se diseño priorizando la claridad y facilidad de uso, con un panel de entrada claro y resultados organizados en secciones facilmente distinguibles.

Web Scraping con BeautifulSoup:

BeautifulSoup es una biblioteca de Python para extraer datos de archivos HTML y XML. VERIFEX la utiliza para obtener el contenido textual de los articulos de noticias, eliminando elementos no deseados como scripts, estilos, navegacion y publicidad, para presentar unicamente el texto relevante al modelo de IA.

2.3.4. Social

El aspecto social de VERIFEX se centra en la lucha contra la desinformacion y el fomento de la alfabetizacion mediatica en la sociedad. La herramienta busca empoderar a los ciudadanos para que puedan tomar decisiones informadas sobre la informacion que consumen y comparten.

Participacion ciudadana:

En Mexico, la desinformacion se ha convertido en un problema significativo que afecta diversos aspectos de la vida social y politica. Segun estudios recientes, mas del 70% de los mexicanos considera que las noticias falsas son un problema grave para el pais. Sin embargo, muchas personas carecen de las herramientas y conocimientos necesarios para identificar contenido falso.

VERIFEX busca abordar esta problematica proporcionando una herramienta accesible que cualquier persona

pueda utilizar para verificar la credibilidad de las noticias que encuentra en línea. Al hacerlo, contribuye a la formación de una ciudadanía más informada y crítica.

Accesibilidad y equidad:

Es importante que la herramienta sea accesible para todos los usuarios, independientemente de su nivel de habilidad técnica. Se busca diseñar una interfaz sencilla y fácil de usar, con instrucciones claras y resultados fáciles de interpretar. VERIFEX está disponible en español e inglés para garantizar un mayor alcance.

Privacidad y Ética:

Una parte importante en el desarrollo de VERIFEX es la protección de la privacidad de los datos de los usuarios. A diferencia de muchas herramientas en línea que envían datos a servidores externos, VERIFEX realiza todo el análisis de forma local en el equipo del usuario. Esto significa que ninguna información sale del equipo, garantizando la privacidad total.

Existe un código de Ética del Ingeniero en Sistemas Computacionales que incluye principios fundamentales como la confidencialidad, honestidad y el compromiso con el bienestar social. VERIFEX se alinea con estos principios al:

- No almacenar ningún dato del usuario (principio de confidencialidad).

- Ser transparente en su funcionamiento (principio de honestidad).

- Proporcionar una herramienta gratuita que beneficia a la sociedad (principio de bienestar social).

- No crear sistemas que comprometan la información de terceros.

2.4. Estudio de mercado

El estudio de mercado es una herramienta fundamental para entender el panorama competitivo y las necesidades de los usuarios. Las herramientas de verificación de noticias han experimentado un crecimiento en los últimos años, impulsadas por la creciente preocupación por la desinformación.

a) Google Fact Check Tools:

Creador: Google LLC

Descripción: Herramienta que permite buscar verificaciones de hechos realizadas por organizaciones de fact-checking en todo el mundo.

Plataforma: Web (requiere conexión a internet)

Limitaciones: Depende de verificaciones externas, no realiza análisis automático.

b) Verificado MX:

Creador: Alianza de medios mexicanos y organizaciones civiles

Descripcion: Iniciativa de fact-checking mexicana que verifica contenido viral y noticias falsas.

Plataforma: Web y redes sociales

Limitaciones: Verificacion manual, tiempo de respuesta limitado.

c) Snopes:

Creador: Snopes Media Group

Descripcion: Plataforma de verificacion de hechos enfocada en desmentir rumores y leyendas urbanas.

Plataforma: Web (gratuita con publicidad)

Limitaciones: Contenido mayormente en ingles, verificacion manual.

d) Politifact:

Creador: Poynter Institute

Descripcion: Plataforma de fact-checking enfocada en verificar declaraciones de figuras politicas.

Plataforma: Web

Limitaciones: Enfoque en politica estadounidense, verificacion manual.

VERIFEX se diferencia de estas herramientas al ofrecer: analisis automatico mediante IA local, funcionamiento sin conexion a internet, privacidad total (sin envio de datos), soporte bilingue espanol/ingles, y disponibilidad gratuita sin publicidad.

2.5. Viabilidad

La viabilidad del proyecto VERIFEX es esencial para determinar su factibilidad y potencial de exito. En esta etapa se realizan evaluaciones exhaustivas en areas clave para asegurar que el proyecto sea viable y beneficioso para los usuarios.

Desde el punto de vista tecnologico, VERIFEX utiliza tecnologias maduras y ampliamente adoptadas: React para el frontend, Python con Flask para el backend, y Ollama para la IA local. Todas estas herramientas cuentan con documentacion extensa, comunidades activas y soporte continuo.

Desde el punto de vista economico, todos los componentes de VERIFEX son de codigo abierto y gratuitos, eliminando costos de licencias. El unico requisito de hardware es una computadora moderna capaz de ejecutar Ollama, lo que la mayoría de los usuarios ya posee.

La viabilidad operativa se analiza considerando la capacidad del equipo de desarrollo para llevar a cabo el proyecto de manera eficiente, con conocimientos en desarrollo web, Python, React e integracion de IA local.

2.5.1. Riesgos

A continuacion se presentan los principales riesgos identificados para el proyecto:

Dependencia del modelo de IA: La precision del analisis depende del modelo de IA utilizado (llama3.2:1b). Modelos mas pequenos pueden tener menor precision que modelos grandes.

Compatibilidad del scraping: Algunos sitios web pueden tener protecciones anti-scraping que impidan la extraccion del contenido.

Rendimiento: El analisis con IA local puede ser lento en equipos sin aceleracion GPU.

Instalacion: La configuracion inicial requiere la instalacion de Node.js, Python y Ollama, lo que puede ser complejo para usuarios no tecnicos.

Actualizaciones: Los cambios en las paginas web analizadas o en las APIs de Google News pueden requerir actualizaciones del software.

2.5.2. Plan de contingencia

Para mitigar los riesgos identificados, se establecen las siguientes estrategias:

Multiples modelos de IA: El sistema intenta con varios modelos (llama3.2:1b, phi3:mini, mistral, llama3) en orden de preferencia hasta obtener una respuesta valida.

User-Agent realista: El scraper utiliza headers de navegador real para evitar bloqueos.

Timeouts y manejo de errores: Se implementan timeouts y manejo de errores en todas las operaciones de red.

Documentacion clara: Se proporcionan guias detalladas de instalacion y solucion de problemas comunes.

Codigo modular: El sistema esta disenado con componentes modulares que facilitan las actualizaciones y el mantenimiento.

2.6. Factibilidad

2.6.1. Capital Humano

El equipo de desarrollo de VERIFEX esta conformado por los siguientes integrantes:

Chapa Tinajero Francisco Yahel: Programador backend - Implementacion del servidor Flask y la integracion con Ollama.

Fernandez Casas Carlos Axel: Programador frontend - Desarrollo de la interfaz de usuario en React y TypeScript.

Gallardo Cortes Valeria: Disenadora UX/UI - Diseno de la interfaz, experiencia de usuario y estetica visual.

Garcia Garcia Jose Armando: Integrador y documentacion - Coordinacion del proyecto, pruebas y documentacion.

2.6.2. Recursos financieros

El proyecto requiere una inversion minima ya que todas las herramientas utilizadas son gratuitas y de codigo abierto. Los costos principales son el tiempo de desarrollo del equipo y los equipos de computo ya existentes.

2.6.3. Recursos Materiales

Para el desarrollo de VERIFEX se requieren los siguientes recursos materiales:

Computadoras con sistema operativo Windows, macOS o Linux.

Conexion a internet para descarga de dependencias y busqueda de noticias similares.

Espacio en disco: aproximadamente 500 MB para las herramientas de desarrollo y 4 GB para el modelo de IA (llama3.2:1b).

CAPITULO 3

METODOLOGIA DE DESARROLLO

3.1. Metodologia Kanban

Para el desarrollo de VERIFEX se ha seleccionado la metodologia Kanban, un enfoque agil para la gestion de proyectos de software que se basa en la visualizacion del flujo de trabajo, la limitacion del trabajo en progreso (WIP) y la mejora continua.

Principios de Kanban:

1. Visualizar el flujo de trabajo: Mediante un tablero Kanban con columnas que representan las etapas del proceso de desarrollo.
2. Limitar el trabajo en progreso (WIP): Establecer limites maximos de tareas en cada columna para evitar la sobrecarga del equipo.
3. Gestionar el flujo: Monitorear y optimizar el movimiento de las tareas a traves del tablero.
4. Hacer las politicas explicitas: Definir claramente las reglas para mover tareas entre columnas.
5. Implementar ciclos de retroalimentacion: Realizar reuniones periodicas para revisar el progreso y mejorar el proceso.
6. Mejorar de forma colaborativa: Fomentar la mejora continua mediante la experimentacion y la colaboracion del equipo.

El tablero Kanban para VERIFEX se organiza en las siguientes columnas:

Backlog: Tareas pendientes por realizar (requerimientos, funcionalidades, mejoras).

Por Hacer (To Do): Tareas priorizadas para el siguiente ciclo de trabajo.

En Progreso (In Progress): Tareas en las que se esta trabajando actualmente (limite: 2 tareas por persona).

En Revision (Review): Tareas completadas pendientes de revision y pruebas.

Terminado (Done): Tareas completamente finalizadas y probadas.

Kanban fue seleccionado sobre otras metodologias debido a que ofrece flexibilidad para adaptarse a los cambios, permite entregas continuas de valor, y es ideal para equipos pequenos donde los roles pueden superponerse. A diferencia de metodologias tradicionales en cascada, Kanban permite ajustar prioridades sobre la marcha sin

interrumpir el flujo de trabajo.

3.2. Fase 1: Gestion del Backlog (Requerimientos)

En esta fase se define y prioriza el backlog del producto, que contiene todos los requerimientos, funcionalidades y mejoras identificadas para VERIFEX.

3.2.1. Levantamiento de requerimientos

El levantamiento de requerimientos se realizo a traves de las siguientes actividades:

- Investigacion de herramientas existentes de fact-checking y deteccion de fake news.
- Identificacion de las limitaciones de las herramientas actuales.
- Analisis de las necesidades del usuario final en terminos de accesibilidad y usabilidad.
- Definicion del alcance del proyecto basado en los recursos disponibles.
- Reuniones del equipo de desarrollo para definir prioridades y funcionalidades clave.

3.2.2. Descripcion del proceso de negocio

El proceso de negocio de VERIFEX se describe de la siguiente manera:

1. El usuario ingresa una URL de una noticia en la interfaz de la aplicacion.
2. El backend recibe la solicitud y verifica que Ollama este funcionando correctamente.
3. El sistema realiza el scraping de la URL proporcionada para extraer el contenido textual del articulo.
4. El contenido extraido se envia al modelo de IA local (Ollama) para su analisis.
5. El modelo de IA clasifica el contenido en una de las categorias predefinidas (REAL, FALSO, SATIRA, ESTAFA, NO VERIFICABLE) y genera un analisis detallado.
6. El backend busca noticias similares en Google News para proporcionar contexto adicional.
7. Los resultados se presentan al usuario en la interfaz de manera clara y organizada.

3.2.3. Historias de usuario

A continuacion se presentan las historias de usuario identificadas para VERIFEX:

HU-01: Analisis de URL:

Como usuario, quiero pegar una URL de una noticia y obtener un analisis de credibilidad para saber si la informacion es confiable.

HU-02: Veredicto claro:

Como usuario, quiero ver un veredicto claro (REAL, FALSO, etc.) para entender rapidamente si la noticia es

confiable.

HU-03: Nivel de confianza:

Como usuario, quiero ver un nivel de confianza numerico para entender que tan seguro es el analisis.

HU-04: Resumen del articulo:

Como usuario, quiero ver un resumen del articulo analizado para entender de que trata sin tener que leerlo completo.

HU-05: Afirmaciones principales:

Como usuario, quiero ver las afirmaciones principales extraidas del articulo para identificar los puntos clave.

HU-06: Analisis detallado:

Como usuario, quiero ver el razonamiento detallado detras del veredicto para entender por que se llego a esa conclusion.

HU-07: Alertas y senales:

Como usuario, quiero ver alertas detectadas y senales positivas para identificar rapidamente aspectos problematicos o confiables.

HU-08: Noticias similares:

Como usuario, quiero ver noticias similares relacionadas para tener contexto adicional sobre el tema.

HU-09: Soporte bilingue:

Como usuario, quiero poder cambiar entre espanol e ingles para usar la herramienta en mi idioma preferido.

HU-10: Privacidad:

Como usuario, quiero que el analisis se realice localmente sin enviar mis datos a internet para proteger mi privacidad.

3.2.4. Requerimientos no funcionales

Los requerimientos no funcionales identificados para VERIFEX son:

Rendimiento: El analisis de una URL no debe tomar mas de 60 segundos en condiciones normales.

Disponibilidad: La aplicacion debe funcionar sin conexion a internet (excepto para la busqueda de noticias similares).

Privacidad: Todos los analisis deben realizarse localmente sin transmision de datos a servidores externos.

Compatibilidad: La aplicacion debe funcionar en navegadores modernos (Chrome, Firefox, Safari, Edge).

Usabilidad: La interfaz debe ser intuitiva y requerir minimo conocimiento tecnico para su uso.

Seguridad: No se debe almacenar ningun dato del usuario en ningun momento.

3.2.5. Requerimientos del sistema

Los principales interesados en adquirir y utilizar VERIFEX son usuarios generales que buscan verificar la credibilidad de noticias en línea. Se espera que la aplicación se adapte a las necesidades específicas identificadas durante la investigación y el desarrollo.

El sistema permitirá a los usuarios analizar URLs de noticias sin necesidad de registrarse ni iniciar sesión. Los usuarios simplemente pegan la URL y hacen clic en Analizar. La aplicación incluirá una interfaz clara que muestra los resultados del análisis de forma organizada y fácil de entender.

Para crear una experiencia inmersiva, se integrará una estética cyberpunk con efectos visuales como animación de cuadrícula, efecto glitch en el título, vignette CRT y líneas de scan. Estos elementos estéticos funcionan para mejorar la experiencia del usuario y reforzar la identidad de la aplicación.

3.2.6. Estudio de usabilidad

El estudio de usabilidad se enfoca en evaluar la experiencia de los usuarios al utilizar VERIFEX para analizar la credibilidad de noticias.

a) Facilidad de uso:

La interfaz de usuario debe ser intuitiva y fácil de usar. La interfaz debe tener un campo de entrada claro, un botón de análisis visible y resultados organizados en secciones bien diferenciadas. Es importante evaluar si los elementos de la interfaz son comprensibles y si las acciones requeridas son sencillas y directas.

b) Rendimiento:

VERIFEX debe proporcionar el análisis de forma rápida, precisa y sin demoras excesivas. Es esencial evaluar la velocidad del análisis en diferentes tipos de contenido y bajo diferentes condiciones de hardware.

c) Accesibilidad y comodidad:

Debemos asegurarnos que la aplicación sea accesible para la audiencia objetivo, evitando que los usuarios encuentren dificultades para comprender los resultados o navegar por la interfaz.

d) Compatibilidad del dispositivo:

VERIFEX debe ser compatible con navegadores web modernos en diferentes sistemas operativos. Es importante verificar que la aplicación se ejecute sin problemas en los navegadores especificados.

3.3. Fase 2: Diseño y Arquitectura

Durante esta fase se diseña la arquitectura del sistema y se crean los diagramas y wireframes que guiarán la implementación.

3.3.1. Arquitectura del sistema

VERIFEX adopta una arquitectura cliente-servidor de dos capas:

Capa de presentacion (Frontend):

Desarrollada con React 18, TypeScript 5 y Vite. Se encarga de la interfaz de usuario, la entrada de datos y la presentacion de resultados. Incluye componentes como UrlInput, VerdictDisplay, ConfidenceBar, RedFlags, SimilarNews y LanguageToggle.

Capa de logica (Backend):

Desarrollada con Python y Flask. Se encarga de recibir las solicitudes de analisis, realizar el scraping de las URLs, comunicarse con Ollama para el analisis de IA, y buscar noticias similares en Google News.

Capa de IA local (Ollama):

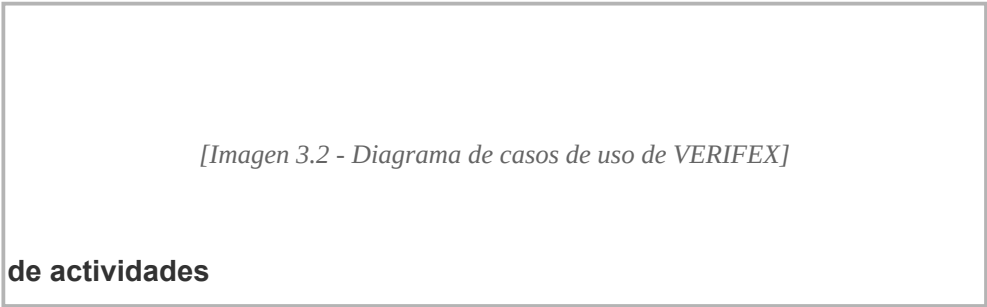
Ollama ejecuta modelos de lenguaje de forma local en el equipo del usuario. Por defecto utiliza llama3.2:1b, pero puede configurarse para usar otros modelos compatibles.



[Imagen 3.1 - Diagrama de arquitectura del sistema VERIFEX]

3.3.2. Diagrama de casos de uso

El diagrama de casos de uso muestra las interacciones entre los actores y las funcionalidades del sistema. Los usuarios pueden ingresar una URL, analizar una noticia, ver resultados detallados, cambiar el idioma y acceder a noticias similares.



[Imagen 3.2 - Diagrama de casos de uso de VERIFEX]

3.3.3. Diagrama de actividades

El diagrama de actividades muestra el flujo completo del proceso de analisis: el usuario ingresa una URL, el sistema verifica el formato, realiza el scraping, envia el contenido a Ollama, procesa la respuesta de la IA, busca noticias similares y presenta los resultados al usuario.



[Imagen 3.3 - Diagrama de actividades del proceso de analisis]

3.3.4. Diagrama de secuencia

El diagrama de secuencia muestra la interaccion temporal entre los diferentes componentes del sistema cuando un usuario solicita un analisis: el frontend envia la URL al backend, el backend realiza el scraping, consulta a Ollama, busca noticias similares en Google News, y retorna los resultados al frontend.

[Imagen 3.4 - Diagrama de secuencia del analisis de URL]

3.3.5. Diagrama de clases

El diagrama de clases muestra las principales clases del sistema, incluyendo las clases del backend (Analyzer, Scraper, NewsFinder) y las interfaces del frontend (Analysis, NewsItem, ApiResponse).

[Imagen 3.5 - Diagrama de clases de VERIFEX]

3.3.6. Diagrama entidad-relacion

El diagrama entidad-relacion muestra las entidades principales del sistema: URL de entrada, contenido extraido, analisis generado, noticias similares y las relaciones entre ellas.

[Imagen 3.6 - Diagrama entidad-relacion de VERIFEX]

3.3.7. Wireframes

Los Wireframes son representaciones graficas que esquematizan la estructura y funcionalidad de la aplicacion mediante bocetos o dibujos.

Wireframe del panel principal:

El Wireframe muestra la vista principal de la aplicacion con el campo de entrada de URL, el boton de analisis, y

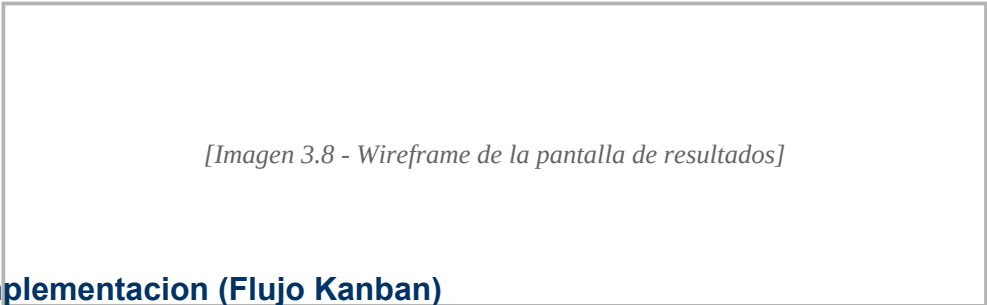
las secciones de resultados organizadas en columnas.



[Imagen 3.7 - Wireframe de la pantalla principal de VERIFEX]

Wireframe de resultados:

El Wireframe muestra la vista de resultados con el veredicto, la barra de confianza, el resumen, las afirmaciones principales, el analisis detallado, las alertas y las noticias similares.



[Imagen 3.8 - Wireframe de la pantalla de resultados]

3.4. Fase 3: Implementacion (Flujo Kanban)

Durante esta fase se implementaron todas las funcionalidades planificadas siguiendo el flujo de trabajo Kanban. Las tareas se movieron a traves del tablero desde el backlog hasta completarse.

Configuracion del entorno de desarrollo:

- Instalacion de Node.js y npm para el frontend.
- Instalacion de Python 3 y pip para el backend.
- Instalacion de Ollama y descarga del modelo llama3.2:1b.
- Configuracion del proyecto con Vite y React.
- Configuracion del servidor Flask con CORS.

Implementacion del backend (Flask + Python):

- Creacion del servidor Flask con rutas para analisis y health check.
- Implementacion del modulo de scraping con BeautifulSoup.
- Implementacion del modulo de analisis con Ollama.
- Implementacion del modulo de busqueda de noticias similares con Google News RSS.
- Integracion de manejo de errores y timeouts.

Implementacion del frontend (React + TypeScript):

- Creacion de la estructura de componentes (App, UrlInput, VerdictDisplay, ConfidenceBar, RedFlags,

SimilarNews, LanguageToggle).

Implementacion de la interfaz de usuario con estetica cyberpunk.

Implementacion de estilos con CSS y Tailwind CSS.

Implementacion de la logica de estados (loading, error, resultados).

Implementacion del soporte bilingue (espanol/ingles).

Integracion con Ollama:

Configuracion del prompt del sistema para el analisis de credibilidad.

Implementacion de la logica de clasificacion (REAL, FALSO, SATIRA, ESTAFA, NO VERIFICABLE).

Implementacion de la verificacion de seguridad para fuentes reconocidas.

Implementacion del fallback a multiples modelos de IA.

3.4.1. Historial de versiones del sistema

A lo largo del desarrollo de VERIFEX se liberaron multiples versiones incrementales, cada una anadiendo funcionalidades especificas. A continuacion se presenta el historial completo de versiones:

v0.1.0 - Scraper de URLs funcional (24/02/2026)

Implementacion del scraper con BeautifulSoup para extraer titulo, descripcion y cuerpo de articulos de noticias a partir de una URL.

v0.2.0 - Analisis con IA via Groq API (07/03/2026)

Integracion con Groq API utilizando el modelo llama-3.3-70b-versatile. Se elimino la dependencia de Ollama para mejorar la precision y velocidad del analisis.

v0.3.0 - Clasificador de credibilidad funcional (25/03/2026)

Implementacion del clasificador con las categorias: REAL, FALSO, SATIRA, ESTAFA y NO VERIFICABLE. Se incorporo el prompt engineering para analisis detallado con niveles de confianza numericos.

v0.4.0 - Frontend conectado al backend (15/04/2026)

Conexion API REST entre React y Flask. Primer renderizado de resultados en la interfaz de usuario. Arquitectura cliente-servidor completamente funcional.

v0.5.0 - UI completa con estados y noticias similares (03/05/2026)

Implementacion de componentes base de UI (UrlInput, VerdictDisplay, ConfidenceBar, RedFlags). Estados de carga, error y resultados. Busqueda de noticias similares via Google News RSS y busqueda semantica.

v0.6.0 - Soporte bilingue ES/EN (09/05/2026)

Implementacion del cambio de idioma espanol/ingles. Traduccion completa de toda la interfaz y textos dinamicos segun el idioma seleccionado.

v0.7.0 - Clasificacion avanzada: tipo de articulo y deteccion de estafas (16/05/2026)

Incorporacion de article_type con categorias: informativa, comercial, opinion, clickbait y denuncia. Deteccion de estafas (is_scam). Badges visuales con colores distintivos para cada tipo.

v0.8.0 - Diseno visual cyberpunk finalizado (25/05/2026)

Estetica visual cyberpunk completa con efectos glitch, scanlines y vignette. Panel de previsualizacion del articulo siempre visible. Footer actualizado a POWERED BY GROQ API.

v1.0.0 - Version estable desplegada en Render (20/07/2026)

Configuracion de produccion con Procfile y gunicorn. Despliegue exitoso en Render con dominio publico. Variables de entorno, CORS y PORT configurados para produccion.

v1.1.0 - Refinamiento y optimizacion del clasificador (05/08/2026)

Pruebas exhaustivas con URLs reales. Evaluacion de precision del clasificador. Correccion de errores y timeouts. Optimizacion de prompts para mejores resultados.

v1.2.0 - Version final de tesis (21/08/2026)

Analisis de casos de prueba y documentacion de resultados. Version estable completa para entrega de tesis con todas las funcionalidades implementadas.

Cada version fue documentada en el tablero Kanban del proyecto y registrada en el repositorio de GitHub mediante etiquetas (tags) correspondientes. El diagrama de Gantt del proyecto (ver Anexo) muestra la linea de tiempo completa de desarrollo con cada uno de estos hitos.

3.5. Fase 4: Codificacion

En esta seccion se muestran las principales clases y modulos implementados para VERIFEX, con una breve descripcion de su funcion.

Backend - analyzer.py:

Modulo principal de analisis que contiene la logica de comunicacion con Ollama, el scraping de URLs y la clasificacion de contenido.

```
def analyze_url(url: str) -> dict:
    # Verifica que Ollama este corriendo
    if not check_ollama():
        return {"error": "Ollama no esta corriendo"}
```

```
# Realiza scraping de la URL
scraped = scrape_url(url)
if "error" in scraped:
    return {"error": scraped["error"]}
# Envia el contenido a Ollama para analisis
prompt = construir_prompt(url, dominio, contenido)
for model in modelos_disponibles:
    raw = call_ollama(prompt, model)
    if raw:
        parsed = parse_response(raw)
        if parsed and "verdict" in parsed:
            return {"analysis": parsed}
```

Backend - app.py:

Servidor Flask que expone las rutas de la API para el frontend.

```
@app.route("/analyze", methods=["POST"])
def analyze():
    data = request.get_json(silent=True)
    url = str(data["url"]).strip()
    result = analyze_url(url)
    similar = find_similar_news(title)
    return jsonify({
        "analysis": result["analysis"],
        "similar_news": similar,
        "url_analyzed": url,
        "domain": result.get("domain", ""),
        "is_credible_source": result.get("is_credible_source", False),
    })
```

Frontend - App.tsx:

Componente principal de React que orquesta toda la interfaz de usuario y la logica de estado.

```
export default function App() {
    const [loading, setLoading] = useState(false)
    const [result, setResult] = useState<ApiResponse | null>(null)
    const handleAnalyze = async (url: string) => {
        setLoading(true)
        const res = await fetch('/analyze', {
            method: 'POST',
            body: JSON.stringify({ url }),
        })
        const data: ApiResponse = await res.json()
        setResult(data)
        setLoading(false)
    }
    return (
        <div>
            <UrlInput loading={loading} onAnalyze={handleAnalyze} />
            {result && <VerdictDisplay verdict={result.analysis.verdict} />}
            {result && <ConfidenceBar score={result.analysis.confidence_score} />}
        </div>
    )
}
```

CAPITULO 4

IMPLEMENTACION Y RESULTADOS

4.1. Fase de pruebas

En esta fase se implementan las pruebas para VERIFEX por parte del equipo de tesis, las cuales se engloban en diferentes pruebas con características diferentes.

4.1.1. Pruebas del sistema

La prueba de sistemas es un tipo de prueba de software que realiza comprobaciones del sistema en su conjunto. Consiste en integrar todos los módulos y componentes individuales del software que se han desarrollado, para comprobar si el sistema funciona conjuntamente como se esperaba.

4.1.1.1. Pruebas de validación:

Son el proceso de revisión que verifica que el sistema de software producido cumple con las especificaciones y logra su cometido. En el caso de VERIFEX, se validó que:

- La aplicación se inicia correctamente en el entorno local.

- El frontend se comunica correctamente con el backend.

- Ollama responde correctamente a las solicitudes de análisis.

- Los resultados se muestran correctamente en la interfaz de usuario.

4.1.1.2. Pruebas de funcionalidad:

El objetivo es verificar la capacidad del sistema para manejar de manera efectiva el proceso de análisis de URLs.

Prueba 1: Análisis de URL válida

Pasos:

- Abrir la aplicación VERIFEX en el navegador.

- Ingresar una URL válida de un artículo de noticias.

- Hacer clic en el botón Analizar.

- Esperar a que el análisis se complete.

Resultado esperado:

La aplicación muestra los resultados del análisis incluyendo veredicto, nivel de confianza, resumen, afirmaciones principales y análisis detallado.

Prueba 2: Manejo de URL invalida

Pasos:

Abrir la aplicacion VERIFEX en el navegador.

Ingresar una URL invalida o inexistente.

Hacer clic en el boton Analizar.

Resultado esperado:

La aplicacion muestra un mensaje de error claro indicando que no se pudo acceder a la URL.

Prueba 3: Cambio de idioma

Pasos:

Abrir la aplicacion VERIFEX en el navegador.

Hacer clic en el boton de cambio de idioma (ES/EN).

Verificar que todos los textos de la interfaz cambien al idioma seleccionado.

Resultado esperado:

La interfaz se actualiza completamente al idioma seleccionado sin necesidad de recargar la pagina.

Prueba 4: Visualizacion de noticias similares

Pasos:

Realizar un analisis exitoso de una URL.

Desplazarse hacia abajo para ver la seccion de Noticias Similares.

Resultado esperado:

La aplicacion muestra una cuadrícula con noticias similares obtenidas de Google News, cada una con titulo, fuente y fecha de publicacion.

4.1.2. Pruebas de usabilidad

Este apartado evalua la usabilidad general de VERIFEX en diferentes escenarios y valida que las características funcionen segun lo esperado.

Objetivos de prueba:

Objetivo de Prueba	Objetivo	Descripcion
Velocidad de procesamiento	El analisis debe completarse en un tiempo razonable	Medir el tiempo que toma desde que se ingresa la URL hasta que se muestran los resultados
Interfaz de Usuario	La interfaz debe ser intuitiva y facil de usar	Evaluar la facilidad de uso, el diseno y la navegacion de la interfaz.

Compatibilidad	Funcionar correctamente en diferentes navegadores	Verificar que VERIFEX funciona correctamente en Chrome, Firefox, Safari y Edge
Claridad de resultados	Los resultados deben ser claros y fáciles de interpretar	El usuario puede comprender el veredicto, el nivel de confianza y la evidencia

CONCLUSIONES

El desarrollo de VERIFEX, un analizador de credibilidad de noticias basado en inteligencia artificial local, se presenta como una solución innovadora para abordar los desafíos asociados con la desinformación y las noticias falsas en el entorno digital. Este enfoque tecnológico, que combina accesibilidad con inteligencia artificial local, tiene el potencial de transformar la manera en que los usuarios verifican la información que consumen.

VERIFEX, al estar diseñado como una aplicación web que funciona de manera local con IA gratuita, se centra en ofrecer una experiencia accesible para cualquier usuario, desde jóvenes hasta adultos. A través de su interfaz intuitiva y sus análisis detallados, los usuarios no solo obtienen una clasificación de credibilidad, sino que también adquieren un entendimiento de por qué un contenido se considera confiable o no. Este método innovador permite superar las limitaciones de las herramientas tradicionales de verificación, que a menudo dependen de verificaciones manuales, conexiones a internet o suscripciones de pago.

El objetivo del proyecto VERIFEX de proporcionar una herramienta accesible y gratuita para la verificación de credibilidad responde a una necesidad crítica de mayor alfabetización mediática y conciencia pública sobre la desinformación. Al ofrecer una herramienta educativa que combina tecnología con accesibilidad, VERIFEX tiene el potencial de mejorar significativamente la capacidad de los usuarios para identificar noticias falsas, contribuyendo así a reducir la propagación de desinformación y fomentar un consumo más crítico de información.

La implementación de VERIFEX representa un avance importante en la utilización de inteligencia artificial local para fines sociales, demostrando que es posible crear herramientas poderosas y accesibles que funcionen completamente en el equipo del usuario sin comprometer la privacidad ni requerir costosas suscripciones. Si el proyecto se promueve eficazmente, VERIFEX no solo informará, sino que también empoderará a los usuarios para enfrentar y combatir la desinformación, marcando una diferencia significativa en la educación mediática y la conciencia pública sobre este problema.

ANEXO A: MANUAL TECNICO

I. Introduccion

Este manual tecnico esta disenado para proporcionar una vision detallada y comprensible de VERIFEX, un analizador de credibilidad de noticias basado en IA local. Proporciona una guia paso a paso sobre como instalar y usar la aplicacion, asi como una descripcion detallada de sus componentes y funcionalidades.

1. Requisitos del sistema

1.1. Hardware necesario

Sistema operativo: Windows 10/11, macOS 10.15+, o Linux.

Procesador: Intel Core i3 o equivalente (se recomienda i5 para mejor rendimiento).

Memoria: 8 GB de RAM (16 GB recomendados para Ollama).

Almacenamiento: 500 MB para herramientas de desarrollo + 4 GB para el modelo de IA.

1.2. Software necesario

Node.js v18, v20 o v22

Python 3.10+

Ollama (motor de IA local)

Navegador web moderno (Chrome, Firefox, Safari, Edge)

1.3. Dependencias del software

React 18 + TypeScript 5 (Frontend)

Vite 5 (Build tool)

Flask 3.0 + Flask-CORS (Backend)

BeautifulSoup 4 + lxml (Web scraping)

Ollama + llama3.2:1b (IA local)

II. Instalacion

2.1. Instalacion de Node.js

Descargar Node.js desde nodejs.org e instalar. Verificar la instalacion con: `node --version`

2.2. Instalacion de Python

Descargar Python desde python.org e instalar. Verificar con: `python3 --version`

2.3. Instalacion de Ollama

Descargar Ollama desde ollama.com e instalar. Luego ejecutar: `ollama pull llama3.2:1b` y `ollama serve`

2.4. Instalacion del backend

Navegar a la carpeta `server` y ejecutar: `pip3 install -r requirements.txt` y `python3 app.py`

2.5. Instalacion del frontend

En la raiz del proyecto ejecutar: `npm install` y `npm run dev`

ANEXO B: MANUAL DE USUARIO

I. Introduccion

Este manual esta disenado para proporcionar una guia detallada sobre las diferentes pantallas y funciones de VERIFEX, con el fin de mejorar la experiencia de usuario y ayudar a aprovechar al maximo todas las caracteristicas de la aplicacion.

1. Objetivo

El objetivo de este manual es proporcionar una referencia completa y accesible sobre las pantallas y funcionalidades de VERIFEX.

2. Requerimientos

Navegador web moderno (Chrome, Firefox, Safari, Edge)

Node.js v18+ instalado

Python 3.10+ instalado

Ollama instalado y corriendo

II. Uso de la aplicacion

Pantalla principal:

Al abrir VERIFEX en el navegador (<http://localhost:5173>), se muestra la pantalla principal con el logo, el campo de entrada para la URL y el boton de Analizar.

Paso 1: Ingresar URL:

Pega la URL de la noticia que deseas analizar en el campo de texto y presiona el boton Analizar o la tecla Enter.

Paso 2: Esperar el analisis:

La aplicacion mostrara una barra de carga animada mientras se realiza el analisis. Este proceso puede tomar entre 15 y 60 segundos dependiendo del hardware.

Paso 3: Interpretar resultados:

Una vez completado el analisis, se mostraran los siguientes resultados:

Veredicto: REAL, FALSO, SATIRA, ESTAFA o NO VERIFICABLE.

Indice de Confiabilidad: Puntuacion del 0 al 100.

Resumen del Artículo: Breve descripción del contenido analizado.

Afirmaciones Principales: Puntos clave extraídos del artículo.

Análisis Detallado: Razón detrás del veredicto.

Alertas Detectadas: Banderas rojas identificadas.

Señales Positivas: Indicadores de credibilidad.

Noticias Similares: Contenido relacionado de Google News.

ANEXO C: CODIFICACION

A continuacion se presenta la codificacion completa de los modulos principales de VERIFEX.

analyzer.py - Modulo principal de analisis

```
import requests
import json
from bs4 import BeautifulSoup
from urllib.parse import urlparse

OLLAMA_URL = "http://localhost:11434/api/generate"
OLLAMA_HEALTH_URL = "http://localhost:11434"

CREDIBLE_DOMAINS = {
    "milenio.com", "eluniversal.com.mx", "reforma.com", "proceso.com.mx",
    "jornada.com.mx", "excelsior.com.mx", "nmas.com.mx",
    "cnn.com", "bbc.com", "reuters.com", "apnews.com",
    "nytimes.com", "theguardian.com", "elpais.com", "infobae.com",
}

def check_ollama() -> bool:
    try:
        requests.get(OLLAMA_HEALTH_URL, timeout=3)
        return True
    except Exception:
        return False

def get_domain(url: str) -> str:
    parsed = urlparse(url)
    domain = parsed.netloc.lower()
    if domain.startswith("www."):
        domain = domain[4:]
    return domain

def scrape_url(url: str) -> dict:
    headers = {"User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)..."}
    try:
        resp = requests.get(url, headers=headers, timeout=15)
        soup = BeautifulSoup(resp.text, "lxml")
        for tag in soup(["script", "style", "nav", "footer"]):
            tag.decompose()
        paragraphs = soup.find_all("p")
        body = " ".join(p.get_text(strip=True) for p in paragraphs[:50] if len(p.get_text(strip=True)) > 40)
        return {"content": f"Texto: {body[:5000]}"}
    except Exception as e:
        return {"error": str(e)}

def analyze_url(url: str) -> dict:
    if not check_ollama():
        return {"error": "Ollama no esta corriendo"}
    scraped = scrape_url(url)
    if "error" in scraped:
        return {"error": scraped["error"]}
    # ... logica de analisis con Ollama ...
    return result
```

app.py - Servidor Flask

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from analyzer import analyze_url
from news_finder import find_similar_news
```

```

app = Flask(__name__)
CORS(app, origins=["http://localhost:5173"])

@app.route("/analyze", methods=["POST"])
def analyze():
    data = request.get_json(silent=True)
    url = str(data["url"]).strip()
    result = analyze_url(url)
    similar = find_similar_news(result.get("title", ""))
    return jsonify({
        "analysis": result["analysis"],
        "similar_news": similar,
        "url_analyzed": url,
        "domain": result.get("domain", ""),
        "is_credible_source": result.get("is_credible_source", False),
        "error": None,
    })

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"})

if __name__ == "__main__":
    app.run(port=5001, debug=True)

```

App.tsx - Componente principal de React

```

import { useState, useCallback, lazy, Suspense } from 'react'
import UrlInput from './components/UrlInput'
import VerdictDisplay from './components/VerdictDisplay'
import ConfidenceBar from './components/ConfidenceBar'
import RedFlags from './components/RedFlags'
import LanguageToggle from './components/LanguageToggle'

const SimilarNews = lazy(() => import('./components/SimilarNews'))

type Lang = 'es' | 'en'

export default function App() {
    const [lang, setLang] = useState<Lang>('es')
    const [loading, setLoading] = useState(false)
    const [result, setResult] = useState(null)
    const [error, setError] = useState(null)

    const handleAnalyze = async (url: string) => {
        setLoading(true)
        try {
            const res = await fetch('/analyze', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ url }),
            })
            const data = await res.json()
            if (!res.ok || data.error) {
                setError(data.error)
            } else {
                setResult(data)
            }
        } catch {
            setError('Error de conexion con el servidor')
        } finally {
            setLoading(false)
        }
    }

    return (
        <div>

```



```
<h1 className="glitch" data-text="VERIFEX">VERIFEX</h1>
<LanguageToggle lang={lang} onToggle={() => setLang(l => l === 'es' ? 'en' : 'es')} />
<UrlInput lang={lang} loading={loading} onAnalyze={handleAnalyze} />
{result && result.analysis && (
  <>
    <VerdictDisplay verdict={result.analysis.verdict} lang={lang} />
    <ConfidenceBar score={result.analysis.confidence_score} lang={lang} />
    <RedFlags redFlags={result.analysis.red_flags} positiveSignals={result.analysis.positive_signals}
  </>
  lang={lang} />
  </>
)}
</div>
)
}
```